DESARROLLO WEB Y APLICACIONES MÓVILES · Despliegue en AWS

Desplegar la API VetCare con MySQL en dos instancias EC2

La API Dart en una subred pública · MySQL 8 en una subred privada · nginx como proxy inverso · los 16 endpoints accesibles desde Internet

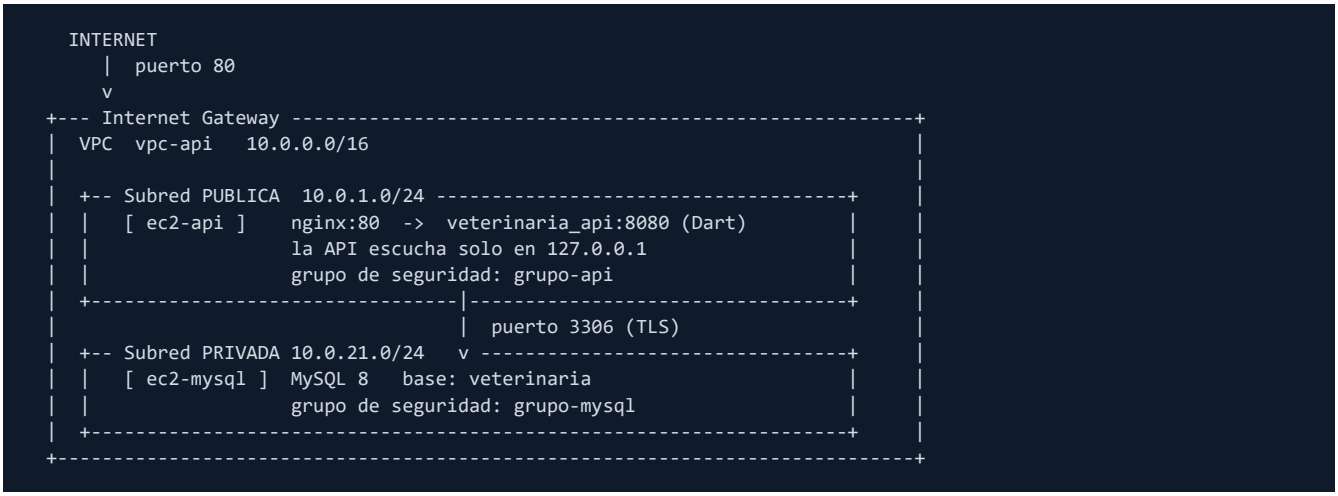
Proyecto	Entorno	Duración estimada	Acceso a las instancias
VetCare · API Caso 2 (E01–E16)	AWS Academy Learner Lab	3 a 4 horas	Session Manager (sin SSH ni claves)

Qué va a lograr. La misma API que corre en su computador (Dart + MySQL local) va a quedar corriendo en AWS, guardando los datos en un MySQL que vive en otra máquina, dentro de una subred sin entrada desde Internet. Al terminar, `http://SU-IP-PUBLICA/api/...` responderá los 16 endpoints (E01–E16), y el script `tool/probar_api.py` del repositorio los recorrerá todos desde su computador para demostrarlo.

Lo que NO hay que cambiar del código. Ni las rutas, ni las validaciones, ni las consultas. La API ya lee los datos de conexión desde variables de entorno (`bin/server.dart`): pasar de su MySQL local a un MySQL en otra máquina es cambiar *dónde* apuntan esas variables. El único ajuste es una línea para que el servidor escuche solo en 127.0.0.1 detrás de nginx (paso 0.4). Además, el archivo `deploy/veterinaria-api.service` del repositorio **no se usa**: exige `mariadb.service` en la misma máquina y con la base separada el servicio no arrancaría. El servicio correcto lo crean los datos de usuario de la Fase 3.

Advertencia de costos. Este despliegue usa un **NAT Gateway**, necesario para que la subred privada pueda instalar MySQL. El NAT **no se detiene** cuando termina la sesión del Learner Lab: sigue cobrando las 24 horas, del orden de un dólar por día. **Al terminar cada sesión ejecute la lista de eliminación de la Fase 9**, o como mínimo elimine el NAT Gateway y libere su IP elástica. La Fase 9 explica cómo mantener la API funcionando hasta el día de la revisión sin pagar el NAT.

La arquitectura — Qué va a construir



Capa	Subred	CIDR	Instancia	IP privada	Puertos
API (Dart)	publica-api	10.0.1.0/24	ec2-api	automática	80 público
Datos (MySQL)	privada-db	10.0.21.0/24	ec2-mysql	10.0.21.10	3306 interno

Por qué un binario compilado y no `dart run`**.** `dart run bin/server.dart` compila el código cada vez que arranca y necesita el código fuente al lado. `dart compile exe` produce un ejecutable nativo que arranca en milisegundos: es lo que corresponde para publicar. nginx va delante porque atiende el puerto 80 (que exige privilegios de root, mientras la API corre como el usuario `ubuntu` sin privilegios) y deja la API escuchando solo en `127.0.0.1`, inalcanzable desde fuera.

Fase 0 · Preparación — 20 minutos

1. Abra el **Learner Lab**, presione **Start Lab** y espere el indicador verde. Anote el crédito disponible.
2. Abra la consola de AWS y confirme la región **N. Virginia (us-east-1)**. No la cambie durante el ejercicio.
3. Tenga a mano el repositorio VetCare (carpeta `backend_api/`) y los dos archivos de `docs/despliegue-aws/datos-de-usuario/`. Su contenido completo también está en los Anexos A y B.

0.4 Ajuste de una línea en server.dart

Abra `backend_api/bin/server.dart` en su computador. Al final de `main()`, reemplace las líneas 33 y 34:

```
# ANTES
  // 0.0.0.0 para que el emulador o un telefono en la misma red lleguen.
  final servidor = await io.serve(handler, InternetAddress.anyIPv4, puerto);

# DESPUES
  // HOST=127.0.0.1 en AWS (detras de nginx); sin HOST sigue siendo 0.0.0.0
  // para que el emulador o un telefono en la misma red lleguen.
  final servidor = await io.serve(handler, _env('HOST', '0.0.0.0'), puerto);
```

Sin la variable `HOST` el comportamiento es el mismo de antes, así que su entorno local y el emulador siguen funcionando. En AWS el `.env` pondrá `HOST=127.0.0.1`. Si tiene Dart instalado, compruebe que no quedó ningún error y suba el cambio al repositorio (lo necesita si va a usar la Opción A de la Fase 5):

```
cd backend_api
dart analyze                  # No issues found!
cd ..
git add backend_api/bin/server.dart
git commit -m "API: direccion de escucha configurable con HOST"
git push
```

Sobre los permisos. En el Learner Lab no se pueden crear roles de IAM. Al lanzar cada instancia hay que seleccionar el perfil que ya existe, en **Detalles avanzados → Perfil de instancia de IAM**: aparece como **LabInstanceProfile** o **LabRole**. Sin él, la instancia no aparecerá en Session Manager y no podrá entrar.

VERIFICACION · Fase 0

- [ ] El laboratorio está iniciado y la región es us-east-1.
- [ ] server.dart usa _env('HOST', '0.0.0.0') en io.serve.
- [ ] El cambio está en GitHub (git log -1 lo muestra) o tiene decidido usar la Opción B de la Fase 5.

Fase 1 · La red — 35 minutos

1.1 VPC y subredes

1. **VPC** → **Sus VPC** → **Crear VPC**. Marque **Solo la VPC**. Nombre **vpc-api**, CIDR IPv4 `10.0.0.0/16`, sin bloque IPv6.
2. **Subredes** → **Crear subred**, en vpc-api. Cree las dos de la tabla, ambas en la misma zona de disponibilidad.

Nombre	Zona	CIDR	Para qué
publica-api	us-east-1a	`10.0.1.0/24`	La API, expuesta a Internet
privada-db	us-east-1a	`10.0.21.0/24`	MySQL, sin exposición directa

3. Seleccione **publica-api** → **Acciones** → **Editar configuración de subred** y marque **Habilitar la asignación automática de direcciones IPv4 públicas**. Guarde. No lo haga en privada-db.

1.2 Internet Gateway y NAT Gateway

1. **Puertas de enlace de Internet** → **Crear**. Nombre **igw-api**. Luego **Acciones** → **Asociar a la VPC** y elija vpc-api. Verifique que quede en estado **Attached**.
2. **Puertas de enlace NAT** → **Crear**. Nombre **nat-api**, subred **publica-api** (va en la pública, no en la privada), conectividad **Pública**, y presione **Asignar IP elástica**. Espere el estado **Disponible** (2 a 3 minutos).

1.3 Tablas de ruteo

1. **Tablas de ruteo** → **Crear rt-publica** en vpc-api. En **Rutas** → **Editar rutas** agregue `0.0.0.0/0` con objetivo **Puerta de enlace de Internet** → **igw-api**. En **Asociaciones de subred** → **Editar** asóciela a **publica-api**.
2. Cree **rt-privada**. Agregue `0.0.0.0/0` con objetivo **Puerta de enlace NAT** → **nat-api**. Asóciela a **privada-db**.
3. Revise que ambas tablas conservan la ruta **10.0.0.0/16** → **local**: es la que comunica la API con la base sin pasar por el NAT.

¿Para qué necesita Internet una subred llamada privada? Para instalar MySQL con apt y para que el agente de Session Manager se comunique con AWS. El NAT deja salir conexiones pero no deja entrar ninguna: nadie desde Internet puede alcanzar esa máquina. **Error común:** una subred que no se asocia explícitamente queda en la tabla principal, que no tiene salida a Internet, y la instancia nunca aparece en Session Manager.

VERIFICACION · Fase 1	
[ ]	La VPC vpc-api existe con CIDR 10.0.0.0/16.
[ ]	Las dos subredes existen con los CIDR indicados y en la misma zona.
[ ]	publica-api asigna IP pública automáticamente; privada-db no.
[ ]	El Internet Gateway está Attached a la VPC.
[ ]	El NAT Gateway está Disponible y en la subred publica-api.
[ ]	rt-publica apunta a igw-api y está asociada a publica-api.
[ ]	rt-privada apunta a nat-api y está asociada a privada-db.

Fase 2 · Grupos de seguridad 20 minutos

Cree los dos grupos primero sin reglas de entrada: grupo-mysql necesita referenciar a grupo-api, y no se puede referenciar un grupo que todavía no existe.

1. **VPC → Grupos de seguridad → Crear**, dos veces, en **vpc-api** y sin tocar las reglas: **grupo-api** y **grupo-mysql**. AWS rechaza nombres que empiecen con *sg-*, por eso estos nombres.
2. Ahora edite las **reglas de entrada** de cada uno según la tabla. En **Origen**, cuando se indique un grupo, escriba **grupo-api** (o el ID que empieza con sg-) y selecciónelo de la lista: no escriba una dirección IP.

Grupo	Tipo	Puerto	Origen	Por qué
grupo-api	HTTP	TCP 80	0.0.0.0/0	Es la única capa que se expone a Internet
grupo-mysql	MYSQL/Aurora	TCP 3306	grupo-api	Solo la máquina de la API puede consultar la base

Por qué referenciar el grupo y no el rango 10.0.1.0/24. Las dos formas funcionan hoy. La diferencia aparece cuando la instancia de la API se reemplaza por otra, o cuando hay varias: referenciar el grupo autoriza un rol, no una dirección, y sigue funcionando sin tocar nada. Leída en voz alta, la regla dice: «la base acepta conexiones de cualquier cosa que sea un servidor de la API». Los grupos son *stateful*: la respuesta de MySQL vuelve sin necesitar una regla de salida extra.

VERIFICACION · Fase 2	
[ ]	grupo-api permite entrada TCP 80 desde 0.0.0.0/0.
[ ]	grupo-mysql permite entrada TCP 3306 con origen el grupo grupo-api (no un CIDR).
[ ]	Ningún grupo tiene el puerto 22 abierto: se usa Session Manager.
[ ]	Las reglas de salida quedaron como vienen por omisión (permitir todo).

Fase 3 · Las dos instancias EC2 30 minutos

Lance **primero la base de datos** y después la API. **EC2 → Instancias → Lanzar instancias**.

Parámetro	Valor para ambas instancias
Imagen (AMI)	Ubuntu Server 24.04 LTS, arquitectura **64 bits (x86)**. No elija Arm: el paquete de Dart que instalan los datos de usuario es para amd64
Tipo	t3.micro (si no está disponible, t2.micro)
Par de claves	Continuar sin un par de claves
Almacenamiento	El que viene por omisión (8 GiB gp3) es suficiente
Perfil de IAM	LabInstanceProfile o LabRole, en Detalles avanzados
Especificación de crédito	Estándar, en Detalles avanzados (evita cobros extra)

Nombre	Subred	IP pública	Grupo	IP privada	Datos de usuario
ec2-mysql	privada-db	Deshabilitar	grupo-mysql	`10.0.21.10`	`userdata-1-mysql.sh`
ec2-api	publica-api	Habilitar	grupo-api	`automática`	`userdata-2-api.sh`

1. En **Configuración de red** → **Editar** elija vpc-api, la subred de la tabla, la opción de IP pública y **Seleccionar un grupo de seguridad existente**.
2. Para fijar la IP privada de ec2-mysql: **Configuración de red** → **Configuración avanzada de red**, campo **IP principal** = `10.0.21.10`. Si no lo encuentra, deje que AWS la asigne y anote el valor: lo necesitará en la Fase 6.
3. Pegue el archivo de datos de usuario completo en **Detalles avanzados** → **Datos de usuario**, al final del formulario, incluida la primera línea `#!/bin/bash`. Cópielo desde el archivo .sh (Anexos A y B), no desde el PDF: copiar del PDF puede romper espacios y saltos de línea.
4. Presione **Lanzar instancia**. Repita los pasos para ec2-api.
5. Espere el estado **En ejecución** con las **dos verificaciones superadas**.

Los scripts tardan. Los datos de usuario corren después de que la instancia aparece disponible: ec2-mysql demora de 2 a 5 minutos y ec2-api de 5 a 8 (descarga el SDK de Dart). Siga el avance con `sudo tail -f /var/log/userdata.log`. Los datos de usuario se ejecutan **una sola vez**: si el registro muestra un error, lo más rápido es terminar la instancia y lanzarla de nuevo.

VERIFICACION · Fase 3	
[ ]	Las dos instancias están En ejecución con 2/2 verificaciones.
[ ]	ec2-mysql NO tiene IP pública; ec2-api sí (anótela).
[ ]	La IP privada de ec2-mysql es 10.0.21.10 (o anotó la que asignó AWS).
[ ]	Ambas aparecen en Session Manager al presionar Conectar.

Fase 4 · Preparar MySQL — 25 minutos

Conéctese a **ec2-mysql**: selecciónela en EC2, presione **Conectar** y use la pestaña **Session Manager**.

4.1 Comprobar la instalación

```
sudo su -
tail -3 /var/log/userdata.log        # USER DATA MYSQL TERMINADO
systemctl is-active mysql            # active
mysql --version                      # mysql  Ver 8.0.x ... for Linux on x86_64
```

4.2 Ejecutar el script de preparación

El script quedó en la máquina. **Léalo antes de ejecutarlo**: hace tres cosas que conviene entender. Si quiere otra clave, cambie la variable `CLAVE` con nano antes de ejecutarlo y use esa misma en el .env de la Fase 6.

```
cat /root/preparar-base.sh           # lealo primero
nano /root/preparar-base.sh          # opcional: cambiar CLAVE (Ctrl+O, Enter, Ctrl+X)
bash /root/preparar-base.sh
```

Qué hace el script	Por qué
Cambia `bind-address` a 0.0.0.0 y reinicia MySQL	Por omisión MySQL solo escucha en 127.0.0.1: ninguna máquina externa puede conectarse aunque el grupo de seguridad lo permita. **Este es el paso que más se olvida.** Quien filtra de verdad es grupo-mysql.
Crea la base `veterinaria` con utf8mb4	utf8mb4 admite tildes, eñes y emojis (nombres de mascotas y dueños). El juego de caracteres se define al crear la base y coincide con el de `sql/schema.sql`.
Crea el usuario `vet_app@'10.0.1.%'` con todos los permisos solo sobre `veterinaria.*`	El usuario solo sirve desde la subred de la API: aunque alguien obtenga la clave, no puede usarla desde otro lugar. Es la misma idea del grupo de seguridad, aplicada dentro del motor. Necesita CREATE porque las tablas se crean desde ec2-api en la Fase 6, pero no tiene permisos sobre otras bases.

Al terminar, el script imprime los datos de conexión. Anótelos: los va a escribir en el archivo `.env` de la otra máquina. La IP privada de esta instancia también la obtiene con `hostname -I`.

4.3 Comprobar que MySQL ofrece conexiones cifradas

```
ss -lntp | grep 3306                # LISTEN ... 0.0.0.0:3306 ... mysqld
mysql -e "SHOW VARIABLES LIKE 'have_ssl';"                          # have_ssl | YES
mysql -e "SELECT user, host, plugin FROM mysql.user WHERE user='vet_app';"
# vet_app | 10.0.1.% | caching_sha2_password
```

Por qué importa el SSL. MySQL 8 autentica por omisión con `caching_sha2_password`, que exige una conexión cifrada para enviar la clave. El driver de la API (`mysql_client`) cifra cuando `DB_SSL=true`, que es el valor por omisión. MySQL 8 en Ubuntu genera sus certificados solo al instalarse, por eso `have_ssl` ya está en YES y no hay que configurar nada más.

VERIFICACION · Fase 4

- [ ] ss -lntp | grep 3306 muestra 0.0.0.0:3306 y no 127.0.0.1:3306.
- [ ] SHOW DATABASES lista la base veterinaria.
- [ ] El usuario vet_app existe con host 10.0.1.%.
- [ ] have_ssl es YES.
- [ ] Anotó la IP privada de esta instancia y la clave de vet_app.

Fase 5 · Subir el código a la EC2 de la API — 25 minutos

Conéctese a **ec2-api** por Session Manager. No hay SSH, así que el código llega por git o pegado en la terminal. Use **una** de las dos opciones.

5.1 Opción A: clonar desde GitHub (si el repositorio es público)

```
sudo su - ubuntu
tail -3 /var/log/userdata.log       # USER DATA API TERMINADO (si no, espere)
git clone https://github.com/RGonzalez1903/VetCare.git ~/VetCare    # use la URL de su repositorio
cp -r ~/VetCare/backend_api/. /opt/api/
cd /opt/api
ls -la                              # bin/ lib/ sql/ tool/ pubspec.yaml pubspec.lock .env.ejemplo
```

Si el repositorio es privado, git pedirá usuario y contraseña, y GitHub ya no acepta la contraseña de la cuenta. Use la Opción B, que no depende de GitHub, en vez de dejar un token de acceso escrito en la instancia.

5.2 Opción B, paso 1: en SU computador, generar el paquete en texto

En PowerShell, dentro de la carpeta raíz del repositorio (la que contiene `backend_api`). El paquete pesa unos 26 KB en texto, lo que cabe pegado en la terminal:

```
# Comprime la API sin archivos generados ni la coleccion de Postman
tar --exclude='backend_api/.dart_tool' `
    --exclude='backend_api/build' `
    --exclude='backend_api/postman' `
    --exclude='backend_api/.DS_Store' `
    -czf api.tgz backend_api

# Lo convierte a texto y lo deja en el portapapeles
[Convert]::ToBase64String([IO.File]::ReadAllBytes("api.tgz")) | Set-Clipboard

Write-Host "Listo: el paquete esta en el portapapeles"
```

5.3 Opción B, paso 2: en la terminal de ec2-api, reconstruirlo

```
sudo su - ubuntu
tail -3 /var/log/userdata.log        # USER DATA API TERMINADO (si no, espere)
cd /opt/api

# Pegue el texto entre las dos lineas FIN (Ctrl+V o clic derecho)
cat > api.b64 <<'FIN'
...aqui va el texto pegado...
FIN

base64 -d api.b64 > api.tgz
tar xzf api.tgz --strip-components=1
rm api.b64 api.tgz
ls -la                               # bin/ lib/ sql/ tool/ pubspec.yaml pubspec.lock .env.ejemplo
```

Si el texto es demasiado largo para pegarlo de una vez, la terminal de Session Manager puede cortarlo y `base64 -d` dará error. En ese caso suba el archivo `api.tgz` a un bucket de S3 desde la consola y descárguelo en `/opt/api` con `aws s3 cp s3://su-bucket/api.tgz .`, y siga con `tar xzf`. El perfil de IAM del laboratorio ya tiene permisos para hacerlo.

```
# Con cualquiera de las dos opciones, confirme que llego el ajuste de la Fase 0:
grep -n "_env('HOST'" bin/server.dart
```

VERIFICACION · Fase 5	
[ ]	/opt/api contiene bin/, lib/, sql/, tool/, pubspec.yaml, pubspec.lock y .env.ejemplo.
[ ]	grep muestra la línea con _env('HOST', '0.0.0.0').
[ ]	Los archivos pertenecen al usuario ubuntu (ls -la lo muestra en la tercera columna).

Fase 6 · Configurar y arrancar la API 35 minutos

6.1 Comprobar lo que instalaron los datos de usuario

```
dart --version                    # Dart SDK version: 3.x.y  (debe ser 3.13.7 o superior)
free -h | grep Swap               # Swap: 1.0Gi
systemctl is-active nginx         # active
```

6.2 Compilar la API

```
cd /opt/api
dart pub get                      # descarga shelf, mysql_client, bcrypt...  Got dependencies!
dart compile exe bin/server.dart -o veterinaria_api     # tarda 1 a 3 minutos
ls -lh veterinaria_api            # un ejecutable de unos 10 MB
```

Si `dart pub get` reclama por la versión del SDK, el pubspec exige Dart 3.13.7 o superior: actualice con `sudo apt-get update && sudo apt-get install --only-upgrade dart`. **Si la compilación termina con *Killed***, faltó memoria: revise que el swap esté activo (`free -h`).

6.3 Crear el archivo .env con los datos de conexión

```
cp .env.ejemplo .env
openssl rand -hex 32              # copie el resultado: sera JWT_SECRET
nano .env
# Ajuste DB_HOST con la IP privada de ec2-mysql (Fase 4),
# DB_PASSWORD con la clave del script de preparacion y JWT_SECRET con lo que genero openssl.
# Guarde con Ctrl+O, Enter, y salga con Ctrl+X.
chmod 600 .env
ls -l .env                        # -rw------- 1 ubuntu ubuntu ... .env
```

Variable	Valor	Para qué
DB_HOST	10.0.21.10	IP privada de ec2-mysql
DB_PORT / DB_NAME	3306 / veterinaria	Puerto del motor y base creada en la Fase 4
DB_USER / DB_PASSWORD	vet_app / su clave	El usuario restringido a 10.0.1.%
DB_SSL	true	Cifra la conexión; MySQL 8 lo exige para caching_sha2_password
HOST / PORT	127.0.0.1 / 8080	La API solo escucha dentro de la máquina; nginx le reenvía
JWT_SECRET	salida de openssl	Firma los tokens de login. Si cambia, los tokens antiguos dejan de servir
ADMIN_EMAIL / ADMIN_CLAVE	admin@clinica.cl / admin12345	Administrador que la API crea en su primer arranque si no hay ninguno. Si los cambia, hágalo **antes** de arrancar: después ya no se vuelve a crear

Por qué un archivo .env y no la clave dentro del código. El código se versiona en git y se comparte; la clave no. Los permisos 600 dejan que solo el usuario `ubuntu` lo lea, y systemd lo carga con `EnvironmentFile`. En un sistema real ni siquiera se usaría un archivo: la clave vendría de AWS Secrets Manager o de SSM Parameter Store, leída con el rol IAM de la instancia.

6.4 Probar la conexión ANTES de arrancar la API

```
mysql -h 10.0.21.10 -u vet_app -p'CambieEstaClave2026' \
      -e "SHOW DATABASES; SHOW STATUS LIKE 'Ssl_cipher';"
# Debe listar veterinaria, y Ssl_cipher con un valor como TLS_AES_256_GCM_SHA384
```

Si este comando falla, deténgase aquí: el problema está en la red o en MySQL, no en la API. Revise en este orden: (1) el bind-address de la Fase 4; (2) la regla 3306 de grupo-mysql con origen grupo-api; (3) que el usuario exista para el rango 10.0.1.%; (4) la IP y la clave que escribió.

6.5 Crear las tablas desde la máquina de la API

```
mysql -h 10.0.21.10 -u vet_app -p'CambieEstaClave2026' < sql/schema.sql
mysql -h 10.0.21.10 -u vet_app -p'CambieEstaClave2026' veterinaria -e 'SHOW TABLES;'
# atenciones  citas  mascotas  recetas  usuarios
```

Observe lo que acaba de pasar. Ese comando se ejecutó en la máquina de la API, pero las tablas se crearon en la otra máquina, la de la subred privada. Puede comprobarlo desde ec2-mysql con `sudo mysql veterinaria -e "SHOW TABLES;"`. El administrador no está en el esquema: lo crea la API en su primer arranque, porque la clave se guarda como hash bcrypt.

6.6 Arrancar la API como servicio

```
sudo systemctl enable --now veterinaria-api
sudo systemctl status veterinaria-api --no-pager | head -5        # Active: active (running)
sudo journalctl -u veterinaria-api -n 20 --no-pager
# Administrador inicial creado: admin@clinica.cl
# API veterinaria escuchando en http://127.0.0.1:8080
ss -lntp | grep -E ':80 |:8080 '
# 127.0.0.1:8080 (veterinaria_api)   y   0.0.0.0:80 (nginx)
```

Qué gana con systemd. `enable` hace que la API arranque sola cada vez que la instancia se enciende (el Learner Lab apaga las instancias al cerrar la sesión), y `Restart=on-failure` la reinicia cada 5 segundos si se cae, por ejemplo si ec2-api enciende antes que ec2-mysql.

6.7 Prueba local, antes de salir a Internet

```
# Directo a la API y a traves de nginx: un 401 aqui es buena noticia (la API responde y pide token)
curl -s http://127.0.0.1:8080/api/mascotas; echo
curl -s http://localhost/api/mascotas; echo
# {"error":"...","codigo":401,...}

# Login del administrador (E01) y una consulta autenticada
TOKEN=$(curl -s http://localhost/api/auth/login -H 'content-type: application/json' \
  -d '{"email":"admin@clinica.cl","clave":"admin12345"}' \
  | sed 's/.*"access_token":"\([^"]*\)".*/\1/')
echo "$TOKEN" | cut -c1-25          # eyJhbGciOiJIUzI1NiIsInR5...
curl -s http://localhost/api/mascotas -H "authorization: Bearer $TOKEN"; echo
```

6.8 Probar los 16 endpoints en la propia instancia

```
cd /opt/api
python3 tool/probar_api.py http://localhost
# OK  200 (esperado 200)  E01 login admin
# ...
# Si cambio la clave del administrador:  ADMIN_CLAVE='su-clave' python3 tool/probar_api.py http://localhost
```

VERIFICACION · Fase 6	
[ ]	El cliente mysql conecta desde ec2-api hacia ec2-mysql y Ssl_cipher tiene valor.
[ ]	SHOW TABLES muestra las 5 tablas en veterinaria.
[ ]	systemctl is-active veterinaria-api responde active.
[ ]	El journal muestra «Administrador inicial creado» y la API escucha en 127.0.0.1:8080.
[ ]	curl con token devuelve JSON; probar_api.py termina con todas las líneas en OK.

Fase 7 · Abrir la API desde Internet — 20 minutos

Este es el objetivo. Copie la **dirección IPv4 pública** de ec2-api desde la consola de EC2 y pruebe desde su computador:

Herramienta	Qué hacer	Qué debe ver
Navegador	Abrir `http://SU-IP/api/mascotas`	El JSON de error 401: la API responde desde Internet y pide token
probar_api.py	En PowerShell, en la raíz del repositorio: `python backend_api\tool\probar_api.py http://SU-IP`	Todas las líneas en OK, de E01 a E16. **Es la evidencia principal de que todos los endpoints funcionan**
Postman	Importar `postman/Veterinaria_API.postman_collection.json`, cambiar la variable `baseUrl` a `http://SU-IP` (sin :8080) y ejecutar primero el login	Las mismas respuestas que en local
App Flutter	`flutter run --dart-define=API_URL=http://SU-IP`	La app inicia sesión y lista datos. El AndroidManifest ya permite HTTP sin cifrar

Escriba http://, no https. No hay certificado configurado. Si el navegador corrige solo a https, borre la «s» y vuelva a intentar. Tampoco agregue :8080: desde Internet solo existe el puerto 80.

Si algo no carga: diagnóstico en orden

Síntoma	Causa más probable	Dónde revisar
502 Bad Gateway	La API no está corriendo o no escucha en 8080	systemctl status veterinaria-api y journalctl -u veterinaria-api -n 30
El journal dice Access denied o no conecta a MySQL	Datos del .env, o la red hacia la base	El .env y el paso 6.4
El journal menciona caching_sha2_password o secure connection	Conexión sin cifrar	DB_SSL=true en el .env y have_ssl=YES (paso 4.3)
No carga nada desde el navegador	Falta la regla 80, o la IP cambió	grupo-api, y la IP pública actual de la instancia
Página 404 de nginx (HTML)	Quedó activo el sitio default de nginx	ls /etc/nginx/sites-enabled/ debe mostrar solo veterinaria
400 «Debe ser una fecha futura» con una fecha que sí lo es	Zona horaria distinta	timedatectl en ambas máquinas debe decir America/Santiago
401 en el login del administrador	Cambió ADMIN_CLAVE después del primer arranque	Use la clave con que arrancó la primera vez

VERIFICACION · Fase 7	
[ ]	http://SU-IP/api/mascotas responde un JSON desde su navegador.
[ ]	probar_api.py contra http://SU-IP termina con todas las líneas en OK.
[ ]	La colección de Postman funciona con baseUrl = http://SU-IP.
[ ]	Capturó la pantalla del resultado de probar_api.py y del navegador con la IP visible.

Fase 8 · Comprobaciones que valen la pena 20 minutos

Estas comprobaciones demuestran qué cambió y qué no al separar la API de la base. Para las de la API use la terminal de ec2-api con el `$TOKEN` del paso 6.7; para las de la base, la de ec2-mysql.

8.1 Los datos están de verdad en la otra máquina

Después de correr probar_api.py (o de crear algo desde Postman), en la terminal de ec2-mysql:

```
sudo mysql veterinaria -e "SELECT id, nombre, email, rol FROM usuarios ORDER BY id DESC LIMIT 5;"
sudo mysql veterinaria -e "SELECT id, nombre, especie, dueno_id FROM mascotas ORDER BY id DESC LIMIT 5;"
sudo mysql veterinaria -e "SELECT id, mascota_id, veterinario_id, fecha_hora, estado FROM citas LIMIT 5;"
```

8.2 La base no es alcanzable desde Internet

Desde PowerShell en su propio computador intente conectarse a la base. Debe fallar: no tiene IP pública ni ruta de entrada.

```
Test-NetConnection 10.0.21.10 -Port 3306          # TcpTestSucceeded : False
```

8.3 Validación en dos capas

```
# En ec2-api: la API rechaza una especie que no existe
curl -s -X POST http://localhost/api/mascotas \
  -H "authorization: Bearer $TOKEN" -H 'content-type: application/json' \
  -d '{"nombre":"Rex","especie":"dragon","sexo":"macho","dueno_id":1}' \
  -w '\n[HTTP %{http_code}]\n'
# -> 400 con "detalles" sobre especie

# En ec2-mysql: la base la rechaza aunque alguien se salte la API
sudo mysql veterinaria -e "INSERT INTO mascotas (nombre,especie,sexo,dueno_id) VALUES ('Malo','dragon','macho',1);"
# -> ERROR 1265 (01000): Data truncated for column 'especie'
```

Dos defensas, no una. La validación de la API rechaza el dato antes de llegar a la base, con un mensaje claro para quien la consume. El ENUM de la tabla, con el modo estricto que MySQL 8 trae activado, lo rechaza aunque alguien entre por otro camino. Si solo existiera la primera, cualquier script que escriba directo en la base podría dejar datos inválidos.

8.4 La integridad referencial se respeta

```
# En ec2-mysql: busque una mascota que tenga citas (las crea probar_api.py)
sudo mysql veterinaria -e "SELECT mascota_id FROM citas LIMIT 1;"

# En ec2-api: intente borrarla con la API (E09). Reemplace ID por el numero obtenido
curl -s -X DELETE http://localhost/api/mascotas/ID \
  -H "authorization: Bearer $TOKEN" -w '\n[HTTP %{http_code}]\n'
# -> 409: la mascota tiene citas

# En ec2-mysql: tampoco se puede saltando la API
sudo mysql veterinaria -e "DELETE FROM mascotas WHERE id = ID;"
# -> ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
```

En el esquema eso está escrito como `ON DELETE RESTRICT` en `fk_citas_mascota`. InnoDB lo aplica sin importar quién intente el borrado.

8.5 Un veterinario no puede tener dos citas a la misma hora

```
# En ec2-mysql: intente duplicar una cita agendada directamente en la base
sudo mysql veterinaria -e "INSERT INTO citas (mascota_id, veterinario_id, fecha_hora, motivo)
  SELECT mascota_id, veterinario_id, fecha_hora, 'duplicada' FROM citas WHERE estado='agendada' LIMIT 1;"
# -> ERROR 1062 (23000): Duplicate entry ... for key 'citas.uq_citas_veterinario_bloque'
```

Por qué funciona. La columna generada `bloque_agendado` vale la fecha solo mientras la cita está agendada, y el índice único sobre (veterinario, bloque) impide repetirla. La API responde 409 en E11, y aunque dos peticiones lleguen al mismo tiempo, solo una puede pasar. Si la consulta no inserta nada, no hay citas agendadas: cree una con E11 desde Postman y repita.

VERIFICACION · Fase 8	
[ ]	Los usuarios, mascotas y citas creados desde Internet aparecen en ec2-mysql.
[ ]	Test-NetConnection a 10.0.21.10:3306 falla desde su computador.
[ ]	La especie inválida da 400 en la API y ERROR 1265 en la base.
[ ]	El borrado de una mascota con citas da 409 en la API y ERROR 1451 en la base.
[ ]	La cita duplicada da ERROR 1062 en la base.

Fase 9 · Eliminar los recursos OBLIGATORIA

Respete el orden: AWS impide borrar un recurso del que dependen otros.

Orden	Recurso	Dónde	Por qué importa
1	Las dos instancias EC2	EC2 → Instancias → Terminar	Liberan el cómputo y la IP pública
2	NAT Gateway	VPC → Puertas de enlace NAT → Eliminar	El más importante: cobra las 24 horas aunque el laboratorio esté detenido
3	IP elástica	VPC → IP elásticas → Liberar	Una IP elástica sin asociar también genera cobro
4	Internet Gateway	Desasociar de la VPC y eliminar	No cobra, pero deja limpio
5	VPC	VPC → Sus VPC → Eliminar	Arrastra subredes, tablas de ruteo y grupos de seguridad

Para mantener la API hasta el día de la revisión sin pagar el NAT. Cuando termine la Fase 8 puede eliminar **solo** el NAT Gateway (paso 2) y liberar su IP elástica (paso 3): la API sigue funcionando, porque ec2-api sale a Internet por el Internet Gateway y habla con la base por la ruta local de la VPC. Lo único que pierde es el acceso por Session Manager a ec2-mysql; para mostrar datos de la base use el cliente desde ec2-api: `mysql -h 10.0.21.10 -u vet_app -p veterinaria`. En rt-privada la ruta 0.0.0.0/0 quedará como *Blackhole*: es normal. Conserve la VPC y las instancias (el Learner Lab las detiene solo) y elimine todo después de la revisión.

Día de la revisión — 15 minutos antes

La IP pública de ec2-api **cambia cada vez que la instancia se detiene y se vuelve a iniciar**. Llegue con esta lista hecha:

VERIFICACION · Antes de presentar	
[ ]	Start Lab y el indicador verde. Las dos instancias En ejecución (si están detenidas, inicie ec2-mysql primero).
[ ]	Anotó la IP pública nueva de ec2-api.
[ ]	En ec2-api: systemctl is-active veterinaria-api nginx responde active dos veces.
[ ]	Desde su computador: python backend_api\tool\probar_api.py http://IP-NUEVA termina todo en OK.
[ ]	Postman con baseUrl actualizado a la IP nueva y el login ya ejecutado.
[ ]	Una pestaña de Session Manager abierta en ec2-api (y en ec2-mysql si mantiene el NAT).
[ ]	Tiene a mano la tabla de endpoints del Anexo C.

Entrega — Contenido del informe

Parte A · Evidencias

- Las dos subredes con sus CIDR, y las dos tablas de ruteo con sus asociaciones.
- Las reglas de entrada de grupo-mysql, donde se vea que el origen es el grupo grupo-api.
- Salida de `ss -lntp | grep 3306` en ec2-mysql.
- Salida de `SHOW TABLES` ejecutado desde ec2-api contra 10.0.21.10 (paso 6.5).
- Salida de `journalctl -u veterinaria-api` con «Administrador inicial creado».
- Resultado de probar_api.py contra http://SU-IP desde su computador, con las 16 operaciones en OK.
- Postman o el navegador con la IP pública visible.
- Resultado del SELECT en ec2-mysql mostrando los datos creados desde Internet (punto 8.1).

Parte B · Preguntas para preparar

1. Describa el camino completo de una petición POST /api/mascotas desde Postman hasta que la fila queda guardada, nombrando en cada salto el componente, el puerto y la regla de grupo de seguridad que lo permite.
2. ¿Qué hubo que modificar del proyecto para pasar del MySQL local a uno en otra máquina, y qué no? Explique por qué las rutas y las validaciones no cambiaron.
3. La subred de la base se llama privada, pero tiene una ruta al NAT Gateway. Explique por qué fue necesaria, por qué no compromete la seguridad, y qué haría distinto en un sistema real (VPC Endpoints).
4. El archivo .env contiene la clave de la base y el secreto JWT en texto plano. Explique qué riesgos tiene y qué mecanismo usaría en producción (Secrets Manager o Parameter Store con el rol IAM).
5. Si mañana la instancia ec2-api se reemplaza por otra con distinta IP privada dentro de la misma subred, ¿habría que cambiar algo en MySQL o en los grupos de seguridad? Justifique con vet_app@'10.0.1.%' y con la regla que referencia a grupo-api.
6. ¿Por qué la API escucha en 127.0.0.1:8080 y no directamente en el puerto 80? ¿Qué pasaría si se abriera el 8080 en grupo-api?

La captura de probar_api.py contra la IP pública es la evidencia principal. Tómela antes de eliminar los recursos: una vez terminada la instancia, la dirección IP deja de existir.

Anexo A · userdata-1-mysql.sh ec2-mysql

Archivo: `docs/despliegue-aws/datos-de-usuario/userdata-1-mysql.sh`. Péguelo completo en Datos de usuario al lanzar ec2-mysql.

```
#!/bin/bash
# Datos de usuario de ec2-mysql (subred privada). Corre una sola vez, como root.
exec > /var/log/userdata.log 2>&1
set -x

# Misma zona horaria que ec2-api: la API compara fechas de citas con la hora local
timedatectl set-timezone America/Santiago

export DEBIAN_FRONTEND=noninteractive
APT="apt-get -o DPkg::Lock::Timeout=300 -y"
$APT update
$APT install mysql-server
systemctl enable --now mysql

# Script de preparacion: se ejecuta a mano en la Fase 4, despues de leerlo
cat > /root/preparar-base.sh <<'EOF'
#!/bin/bash
set -e
CLAVE='CambieEstaClave2026'     # la misma que pondra en DB_PASSWORD del .env
RED_API='10.0.1.%'              # subred publica-api
CNF=/etc/mysql/mysql.conf.d/mysqld.cnf

# 1) Escuchar en todas las interfaces (por omision solo 127.0.0.1)
sed -i 's/^bind-address.*/bind-address = 0.0.0.0/' "$CNF"
systemctl restart mysql

# 2) Base veterinaria en utf8mb4 y 3) usuario vet_app solo desde la subred de la API
mysql <<SQL
CREATE DATABASE IF NOT EXISTS veterinaria
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER IF NOT EXISTS 'vet_app'@'${RED_API}' IDENTIFIED BY '${CLAVE}';
GRANT ALL PRIVILEGES ON veterinaria.* TO 'vet_app'@'${RED_API}';
FLUSH PRIVILEGES;
SQL

echo "---- Datos de conexion para el .env de ec2-api ----"
echo "DB_HOST=$(hostname -I | awk '{print $1}')"
echo "DB_PORT=3306"
echo "DB_USER=vet_app"
echo "DB_PASSWORD=${CLAVE}"
echo "DB_NAME=veterinaria"
echo "---- MySQL escuchando en: ----"
ss -lntp | grep 3306
EOF
chmod 700 /root/preparar-base.sh
echo "USER DATA MYSQL TERMINADO"
```

Anexo B · userdata-2-api.sh ec2-api

Archivo: `docs/despliegue-aws/datos-de-usuario/userdata-2-api.sh`. Péguelo completo en Datos de usuario al lanzar ec2-api.

```
#!/bin/bash
# Datos de usuario de ec2-api (subred publica). Corre una sola vez, como root.
exec > /var/log/userdata.log 2>&1
set -x

# Misma zona horaria que ec2-mysql: la API compara fechas de citas con la hora local
timedatectl set-timezone America/Santiago

export DEBIAN_FRONTEND=noninteractive
APT="apt-get -o DPkg::Lock::Timeout=300 -y"
$APT update
$APT install nginx mysql-client git wget gpg apt-transport-https python3

# --- Dart SDK desde el repositorio oficial de Google ---
wget -qO- https://dl-ssl.google.com/linux/linux_signing_key.pub \
  | gpg --dearmor -o /usr/share/keyrings/dart.gpg
REPO=https://storage.googleapis.com/download.dartlang.org/linux/debian
echo "deb [signed-by=/usr/share/keyrings/dart.gpg arch=amd64] $REPO stable main" \
  > /etc/apt/sources.list.d/dart_stable.list
$APT update
$APT install dart

# --- 1 GB de swap: t3.micro tiene 1 GB de RAM y compilar Dart la necesita ---
fallocate -l 1G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab

# --- Carpeta de la API y plantilla del .env ---
mkdir -p /opt/api
cat > /opt/api/.env.ejemplo <<'EOF'
DB_HOST=10.0.21.10
DB_PORT=3306
DB_USER=vet_app
DB_PASSWORD=CambieEstaClave2026
DB_NAME=veterinaria
DB_SSL=true
HOST=127.0.0.1
PORT=8080
JWT_SECRET=reemplace-por-la-salida-de-openssl-rand-hex-32
ADMIN_EMAIL=admin@clinica.cl
ADMIN_CLAVE=admin12345
EOF
chown -R ubuntu:ubuntu /opt/api
# --- Servicio systemd (se activa en la Fase 6, cuando exista el binario) ---
cat > /etc/systemd/system/veterinaria-api.service <<'EOF'
[Unit]
Description=API Veterinaria VetCare (Dart)
After=network-online.target
Wants=network-online.target
StartLimitIntervalSec=0

[Service]
User=ubuntu
WorkingDirectory=/opt/api
EnvironmentFile=/opt/api/.env
ExecStart=/opt/api/veterinaria_api
Restart=on-failure
```

```
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
systemctl daemon-reload

# --- nginx: puerto 80 publico -> API en 127.0.0.1:8080 ---
cat > /etc/nginx/sites-available/veterinaria <<'EOF'
server {
    listen 80 default_server;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF
ln -sf /etc/nginx/sites-available/veterinaria /etc/nginx/sites-enabled/veterinaria
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx

echo "USER DATA API TERMINADO"
```

Anexo C · Los 16 endpoints — Lo que se va a revisar

ID	Método y ruta	Quién puede usarlo
E01	`POST /api/auth/login`	Público
E02	`POST /api/auth/registro`	Público (el rol siempre es dueno)
—	`GET /api/auth/yo`	Autenticado (destino del Location de E02)
E03	`POST /api/usuarios`	Administrador
E04	`GET /api/veterinarios`	Personal
E05	`GET /api/mascotas`	Autenticado (dueño: solo las suyas)
E06	`POST /api/mascotas`	Recepcionista, administrador
E07	`GET /api/mascotas/{id}`	Dueño de la mascota, personal
E08	`PATCH /api/mascotas/{id}`	Recepcionista, administrador
E09	`DELETE /api/mascotas/{id}`	Administrador (409 si tiene citas)
E10	`GET /api/citas`	Autenticado (dueño: solo las suyas)
E11	`POST /api/citas`	Recepcionista, administrador
E12	`GET /api/citas/{id}`	Dueño de la mascota, personal
E13	`POST /api/citas/{id}/cancelacion`	Recepcionista, administrador
E14	`POST /api/citas/{id}/atencion`	Veterinario asignado
E15	`GET /api/citas/{id}/atencion`	Dueño, veterinario, administrador
E16	`PATCH /api/citas/{id}/atencion`	Veterinario asignado

Los errores tienen el formato `{"error": "...", "codigo": 400, "detalles": {...}}`. Orden de verificación: 401 → 403 por rol → 404 → 403 por pertenencia → 400 → 422/409.